



10 SQL QUESTIONS EVERY BEGINNER MUST KNOW



Master the basics.
Build the **foundation**.

1



Write a query to select all columns from the **Employees** table.

2



Write a query to select the **first_name** and **last_name** from the **Employees** table.

3



Write a query to select all unique **departments** from the **Employees** table.

4



Write a query to select all employees whose **salary** is greater than **50000**.

5



Write a query to select all employees whose **hire_date** is after **'2023-01-01'**.

6



Write a query to sort all employees by **salary** in ascending order.

7



Write a query to sort all employees by **salary** in descending order.

8



Write a query to **count** the total number of employees.

9



Write a query to find the **maximum salary** from the **Employees** table.

10



Write a query to find the **minimum salary** from the **Employees** table.



Tip:
Focus on **understanding**,
not just writing.



Goal:
Solve it **yourself** first,
then compare with AI.



20 INTERMEDIATE SQL QUESTIONS

REAL 3-5 YOE STYLE



You know the basics. Now it's time to think like an analyst.

1



You need to remove duplicates. Would you use **DISTINCT**, **GROUP BY**, **ROW_NUMBER()**, or **CTE**? Explain when each is appropriate.

2



Write a query using **INNER JOIN**. Now optimize it for a table containing **500 million rows**.

3



Your query is taking **12 minutes** to execute. Walk me through how you would **optimize** it.

4



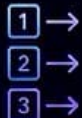
What's the difference between **WHERE** and **HAVING**? Give a business example.

5



When would you use **UNION** instead of **UNION ALL**? Which **performs better** and why?

6



Explain the **execution order** of an SQL query (FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT).

7



Find duplicate records. Then explain why **DISTINCT** is **not** always the right solution.

8



Which is better for deduplication: **ROW_NUMBER()**, **RANK()**, or **DENSE_RANK()**? Why?

9



How would you identify **missing records** between two tables?

10



Find customers who ordered in **January** but not **February**.

11



Explain **LEFT JOIN** vs **NOT EXISTS** vs **NOT IN**. Which would you choose and why?

12



What's the difference between **RANK()**, **DENSE_RANK()**, and **ROW_NUMBER()**? When would each be used?

13



How would you calculate a **running total** without using Window Functions?

14



Write a query to find the **Top 3** employees per department.

15



Find employees earning **more** than their manager.

16



Explain the difference between **Clustered** and **Non-Clustered Indexes**. When can an index actually **slow** a query?

17



Given two different solutions, how would you determine which query is **faster**?

18



What causes **Shuffle** or excessive data movement during joins in large datasets, and how would you **reduce** it?

19



Would you solve this using a **CTE**, **Subquery**, or **Temporary Table**? Why?

20



The interviewer says, "Your query works, but how would it perform on a table with **1 billion rows**?" What changes would you make?



Pro Tip: It's not just about writing SQL. It's about writing the **right SQL**. Think, optimize, then code.



Goal: Write better queries. Think deeper. **Get selected**.



10 SQL QUESTIONS

THAT SEPARATE SENIOR DATA ENGINEERS FROM EVERYONE ELSE



- 21

You have a query that runs in **25 minutes** on a **2-billion-row** table. Without changing the business logic, how would you reduce it to **under 2 minutes**?
- 22

You're joining two huge tables. One has **3 billion rows** and the other has **15 million rows**. How would you **optimize the join**?
Discuss: **Partitioning**, **Broadcast Join**, **Indexes**, **Predicate Pushdown**, **Data Skew**.
- 23

Your SQL query suddenly became **10× slower** overnight, but nobody changed the SQL. How would you investigate the **root cause**?
- 24

Given the same requirement, when would you choose: **Window Functions**, **GROUP BY**, or **Self Join**?
Explain the trade-offs.
- 25

Your pipeline is producing duplicate records. Would you solve it: **Upstream**, **During ETL**, **Inside SQL**, or **In the Data Warehouse**?
Why?
- 26

Design a SQL solution that generates a daily customer snapshot without scanning the full table every day.
How would you make it **incremental**?
- 27

You need to calculate customer lifetime value for **200 million customers**. How would you design the SQL so it remains **scalable**?
- 28

Explain a real situation where **NOT EXISTS** performs better than **LEFT JOIN**. When would you prefer one over the other?
- 29

You're reviewing another engineer's SQL. What are the **five things** you immediately check before approving it for production? (Hint: **correctness**, **readability**, **performance**, **scalability**, **maintainability**.)
- 30

Design the SQL logic for this scenario:
Every night, millions of new transactions arrive. **Update** existing customers, **insert** new customers, **maintain** historical versions, **avoid** duplicates, and make the process **restartable** if it fails halfway.
Explain your complete approach.



Pro Tip: Think beyond writing SQL. Focus on **performance**, **scalability** and real-world impact.



Goal: Solve complex problems. Design scalable solutions. Make data work for the **business**.